

Telco Customer Churn Prediction using Traditional ML Classification Algorithms

In [1]:

```
#Import required libraries:
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

from sklearn.model_selection import train_test_split, cross_validate , StratifiedKFold, K
Fold
from sklearn.preprocessing import LabelEncoder , StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier , ExtraTreesClass
ifier
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

from sklearn.compose import ColumnTransformer

from imblearn.pipeline import Pipeline
#Ignore all incoming warnings (if any):
import warnings
warnings.filterwarnings("ignore")

#Import the dataset:
df = pd.read_csv("dataset.csv")
```

In [2]:

```
#Inspect the dataset:
df.head()
```

Out[2]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

5 rows x 21 columns

In [3]:

```
#Inspect the data:
print(df.info(),end="\n\n\n\n")
print(df.isnull().sum(),end="\n\n\n\n")
```

```
print(df.describe(),end="\n\n\n\n")
print(df.duplicated().sum(),end="\n\n\n\n")
print(df.shape,end="\n\n")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                7043 non-null   object
2   SeniorCitizen         7043 non-null   int64
3   Partner               7043 non-null   object
4   Dependents            7043 non-null   object
5   tenure                7043 non-null   int64
6   PhoneService          7043 non-null   object
7   MultipleLines         7043 non-null   object
8   InternetService       7043 non-null   object
9   OnlineSecurity        7043 non-null   object
10  OnlineBackup          7043 non-null   object
11  DeviceProtection      7043 non-null   object
12  TechSupport           7043 non-null   object
13  StreamingTV           7043 non-null   object
14  StreamingMovies       7043 non-null   object
15  Contract              7043 non-null   object
16  PaperlessBilling      7043 non-null   object
17  PaymentMethod         7043 non-null   object
18  MonthlyCharges        7043 non-null   float64
19  TotalCharges          7043 non-null   object
20  Churn                 7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None
```

```
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

```
SeniorCitizen      tenure  MonthlyCharges
count  7043.000000  7043.000000  7043.000000
mean    0.162147    32.371149    64.761692
std     0.368612    24.559481    30.090047
min     0.000000     0.000000    18.250000
25%     0.000000     9.000000    35.500000
50%     0.000000    29.000000    70.350000
75%     0.000000    55.000000    89.850000
max     1.000000    72.000000   118.750000
```

0

(7043, 21)

In [4]:

```
#Inspect the target variable:
df.Churn.value_counts()
```

Out[4]:

```
Churn
No      5174
Yes     1869
Name: count, dtype: int64
```

In [5]:

```
df.columns
```

Out[5]:

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

In [6]:

```
for i in df.columns:
    print("\n\nNAME = ")
    print(df[i].value_counts())

print("\n\n\n",df.shape)
```

```
NAME =
customerID
3186-AJIEK      1
7590-VHVEG      1
5575-GNVDE      1
8775-CEBBJ      1
2823-LKABH      1
..
6713-OKOMC      1
1452-KIOVK      1
9305-CDSKC      1
9237-HQITU      1
7795-CFOCW      1
Name: count, Length: 7043, dtype: int64
```

```
NAME =
gender
Male      3555
Female    3488
Name: count, dtype: int64
```

```
NAME =
SeniorCitizen
0      5901
1     1142
Name: count, dtype: int64
```

```
NAME =
```

```
Partner
No      3641
Yes     3402
Name: count, dtype: int64
```

```
NAME =
Dependents
No      4933
Yes     2110
Name: count, dtype: int64
```

```
NAME =
tenure
1      613
72     362
2      238
3      200
4      176
...
28      57
39      56
44      51
36      50
0       11
Name: count, Length: 73, dtype: int64
```

```
NAME =
PhoneService
Yes     6361
No       682
Name: count, dtype: int64
```

```
NAME =
MultipleLines
No              3390
Yes            2971
No phone service    682
Name: count, dtype: int64
```

```
NAME =
InternetService
Fiber optic    3096
DSL            2421
No             1526
Name: count, dtype: int64
```

```
NAME =
OnlineSecurity
No              3498
Yes            2019
No internet service    1526
Name: count, dtype: int64
```

```
NAME =
OnlineBackup
No              3088
Yes            2429
No internet service    1526
Name: count, dtype: int64
```

```
NAME =
DeviceProtection
No              3095
Yes            2422
```

No internet service 1526
Name: count, dtype: int64

NAME =
TechSupport
No 3473
Yes 2044
No internet service 1526
Name: count, dtype: int64

NAME =
StreamingTV
No 2810
Yes 2707
No internet service 1526
Name: count, dtype: int64

NAME =
StreamingMovies
No 2785
Yes 2732
No internet service 1526
Name: count, dtype: int64

NAME =
Contract
Month-to-month 3875
Two year 1695
One year 1473
Name: count, dtype: int64

NAME =
PaperlessBilling
Yes 4171
No 2872
Name: count, dtype: int64

NAME =
PaymentMethod
Electronic check 2365
Mailed check 1612
Bank transfer (automatic) 1544
Credit card (automatic) 1522
Name: count, dtype: int64

NAME =
MonthlyCharges
20.05 61
19.85 45
19.95 44
19.90 44
20.00 43
..
56.85 1
101.70 1
48.40 1
108.35 1
72.00 1
Name: count, Length: 1585, dtype: int64

NAME =
TotalCharges
20.2 11
11

```
19.75      9
19.65      8
19.9       8
..
1990.5     1
7362.9     1
346.45     1
306.6      1
108.15     1
Name: count, Length: 6531, dtype: int64
```

```
NAME =
Churn
No      5174
Yes     1869
Name: count, dtype: int64
```

```
(7043, 21)
```

In [7]:

```
#Prepare the data:

cols = [
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "MultipleLines"
]

df[cols] = df[cols].replace("No internet service", "No")

df.drop('customerID', axis=1, inplace=True)

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

df.dropna(inplace=True)

print(df["Churn"].head(5))
df["Churn"] = df["Churn"].replace("No",0)
df["Churn"] = df["Churn"].replace("Yes",1)
```

```
0      No
1      No
2      Yes
3      No
4      Yes
Name: Churn, dtype: object
```

In [8]:

```
#Inspect the shape once again:
df.shape
```

Out[8]:

```
(7032, 20)
```

Now implement Machine Learning:

In [9]:

```
X = df[["gender", "tenure", "Dependents",
        "SeniorCitizen", "Partner",
```

```

        "InternetService", "PhoneService",
        "TechSupport", "Contract",
        "PaymentMethod", "TotalCharges",
        "StreamingTV", "StreamingMovies"]])

y = df['Churn']

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

num_cols = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X_train.select_dtypes(include=['object']).columns.tolist()

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), cat_cols)
])

# All Classification Pipelines:
lr_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', LogisticRegression(max_iter=1000, random_state=42))
])

knn_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', KNeighborsClassifier(n_neighbors=8, n_jobs=-1))
])

et_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', ExtraTreesClassifier(n_estimators=130, max_depth=9, n_jobs=-1, random_state=42))
])

ada_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=6),
        n_estimators=200,
        learning_rate=0.8,
        random_state=42
    ))
])

xgb_pipeline = Pipeline([
    ('preprocessing', preprocessor),
    ('model', XGBClassifier(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1,
        tree_method='hist',
        eval_metric='logloss'
    ))
])

# Cross Validation:

```

```

## Model Dictionary:

models = {
    "Logistic Regression": lr_pipeline,
    "KNN Classifier": knn_pipeline,
    "Extra Trees": et_pipeline,
    "AdaBoost": ada_pipeline,
    "XGBoost": xgb_pipeline
}

kf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)    # Since our dataset is
imbanced.
scoring = ['accuracy', 'f1_weighted']

results = []

for name, model in models.items():
    print(f"Running {name}...")

    cv_results = cross_validate(
        model,
        X_train,
        y_train,
        cv=kf,
        scoring=scoring,
        n_jobs=-1
    )

    results.append({
        "Model": name,
        "CV Accuracy": round(np.mean(cv_results['test_accuracy']), 4),
        "CV F1 Score": round(np.mean(cv_results['test_f1_weighted']), 4)
    })

results_df = pd.DataFrame(results).sort_values(by="CV Accuracy", ascending=False)
print("\n" + "="*50)
print(results_df)

```

```

Running Logistic Regression...
Running KNN Classifier...
Running Extra Trees...
Running AdaBoost...
Running XGBoost...

```

```

=====

```

	Model	CV Accuracy	CV F1 Score
0	Logistic Regression	0.7984	0.7920
2	Extra Trees	0.7950	0.7866
3	AdaBoost	0.7829	0.7761
1	KNN Classifier	0.7780	0.7652
4	XGBoost	0.7737	0.7678

In [10]:

```

#Now make predictions using the best model:

single_data = pd.DataFrame([
    "gender": "Male",
    "Partner": "No",
    "Dependents": "Yes",
    "SeniorCitizen": 0,
    "tenure": 22,
    "PhoneService": "Yes",
    "InternetService": "Fiber optic",
    "TechSupport": "No",
    "Contract": "Month-to-month",
    "PaymentMethod": "Credit card (automatic)",
    "TotalCharges": 1949.4,
    "StreamingTV": "Yes",
    "StreamingMovies": "No"
])

```



```
best_model = models[results_df.iloc[0]["Model"]]
best_model.fit(X_train, y_train)
```

Out[10]:

```

  Pipeline
  i
  preprocessing:
  ColumnTransformer
  ?
  num
  StandardScaler
  ?
  cat
  OneHotEncoder
  ?
  LogisticRegression
  ?
```

In [11]:

```
pred = best_model.predict(single_data)

if pred==0:
    print("This customer will not churn")
else:
    print("This customer will churn")
```

This customer will not churn

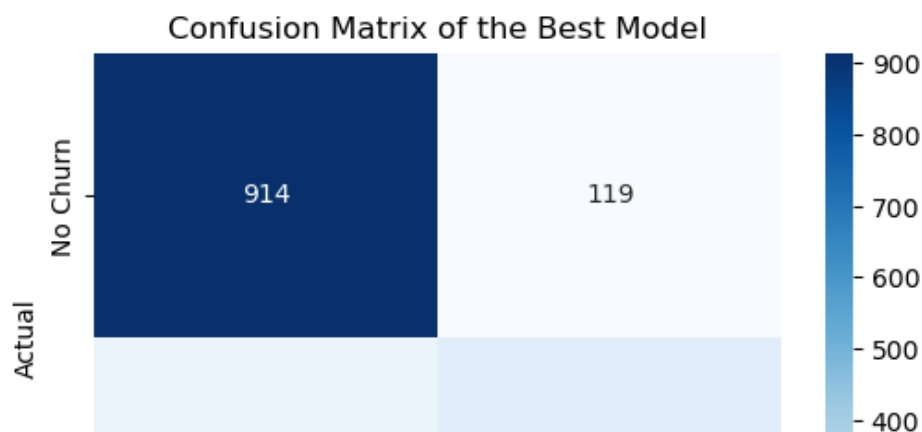
In [12]:

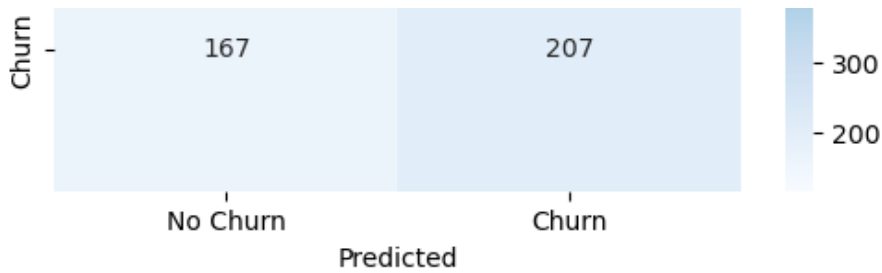
```
y_pred = best_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix of the Best Model")
plt.show()
```





In [13]:

```
# Classification Report
print("Classification Report of the best model:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["No Churn", "Churn"]
))
```

Classification Report of the best model:

	precision	recall	f1-score	support
No Churn	0.85	0.88	0.86	1033
Churn	0.63	0.55	0.59	374
accuracy			0.80	1407
macro avg	0.74	0.72	0.73	1407
weighted avg	0.79	0.80	0.79	1407

In [14]:

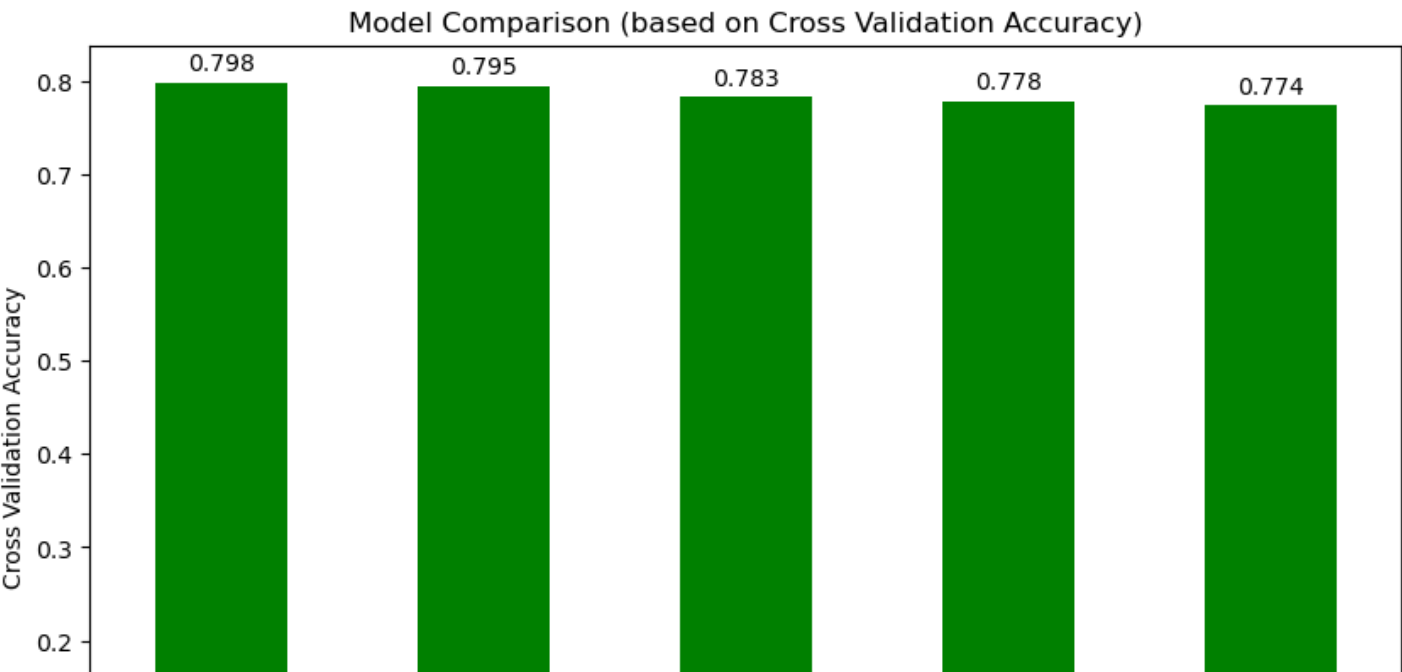
```
#Model comparison:

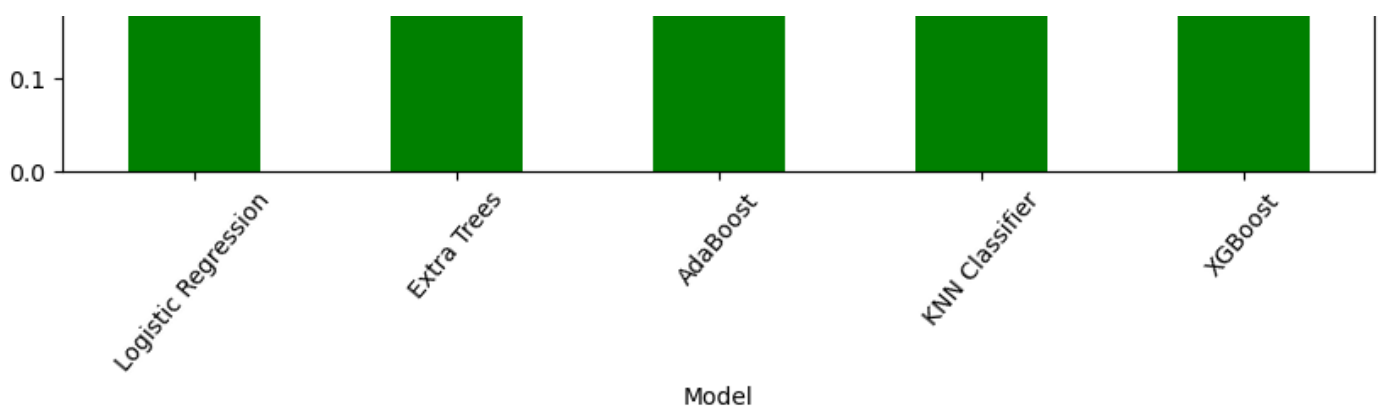
plt.figure(figsize=(10,6))
ax = results_df.set_index("Model")["CV Accuracy"].plot(kind='bar',color="green")
ax.bar_label(ax.containers[0], fmt="%.3f", padding=3)

plt.title("Model Comparison (based on Cross Validation Accuracy)")

plt.xlabel("Model")
plt.ylabel("Cross Validation Accuracy")
plt.xticks(rotation=50)
plt.show()

#Export this model:
import joblib
joblib.dump(best_model, "CustomerChurnPredictionModel.pkl")
```





Out[14]:

```
['CustomerChurnPredictionModel.pkl']
```

In []:

In []:

PROJECT BY:

- **Name:** Om Satyawan Pathak
- **Email:** omsatyawanpathakgit@gmail.com (or) omsatyawanpathakwebdevelopment@gmail.com
- **Linkedin:** www.linkedin.com/in/om-satyawan-pathak-029b02368